

Student Record System – Project Documentation

Group: Group 22
Course: COS201 Computer Programming 1
Project: Student Record System (C)
Language: C (ANSI C / C11)

Table of Contents

- 1. Group Details
- 2. Requirements
- 3. Planning and Considerations
- 4. Program Flow and User Story
- 5. Program Code
- 6. Quick Code Explanation
- 7. Conclusion

1. Group Details

Group Number: Group 22

Team Members:

Student Name	Student ID	School Email
Livingstone Joseph Oboch	30152345	livingstone.obochi@miva.edu.ng
Idowu Joshua Oluwadamilare	30161942	idowu.oluwadamilare@miva.edu.ng
Olalere Isaiah	30114413	olalere.isaiah@miva.edu.ng
Raheem Mujeeb Olamilekan	30093831	mujeeb.raheem@miva.edu.ng
Akinniyi Adeleke Solomon	30132801	adeleke.akinniyi@miva.edu.ng
Cherechi Udensi	30170147	cherechi.udensi@miva.edu.ng
Daniel Abiodun	30153666	daniel.abiodun1@miva.edu.ng
Ayodeji Victor Adeyanju	30159201	ayodeji.victor@miva.edu.ng
Augustine Adolphus Fubara	30169254	augustine.fubara@miva.edu.ng

2. Requirements

Project Title

Student Record System

Project Brief

Create a simple Student Record System in C that allows users to manage student information. The program should include the following functionality:

Initial Setup

- Create a new C project using JetBrains CLion (or preferred development environment)
- Write a program that displays a welcome message and prompts the user to enter their name
- The program should greet the user by their name

Data Storage

- Implement storage for student information, including name, roll number, and marks, using appropriate data types

Input and Output

- Add a feature to input a student's marks and display whether the student has passed or failed based on a passing threshold (marks above 40 are considered passing)

Student Records Management

- Implement a loop that allows the user to input information for multiple students until the user decides to exit
- Create an array to store information for multiple students, with the ability to add, remove, and modify student records
- Implement functions for adding, modifying, and displaying student records
- Ensure functions take appropriate arguments and return relevant information

Memory Management

- Use dynamic memory allocation for student records
- Ensure memory is freed when no longer needed

File Operations

- Implement the ability to save student records to a text file
- Implement the ability to load student records from a text file
- Store information such as name, roll number, and marks in files

Search Functionality

- Allow users to search for a student by their roll number
- Display the relevant information when a student is found

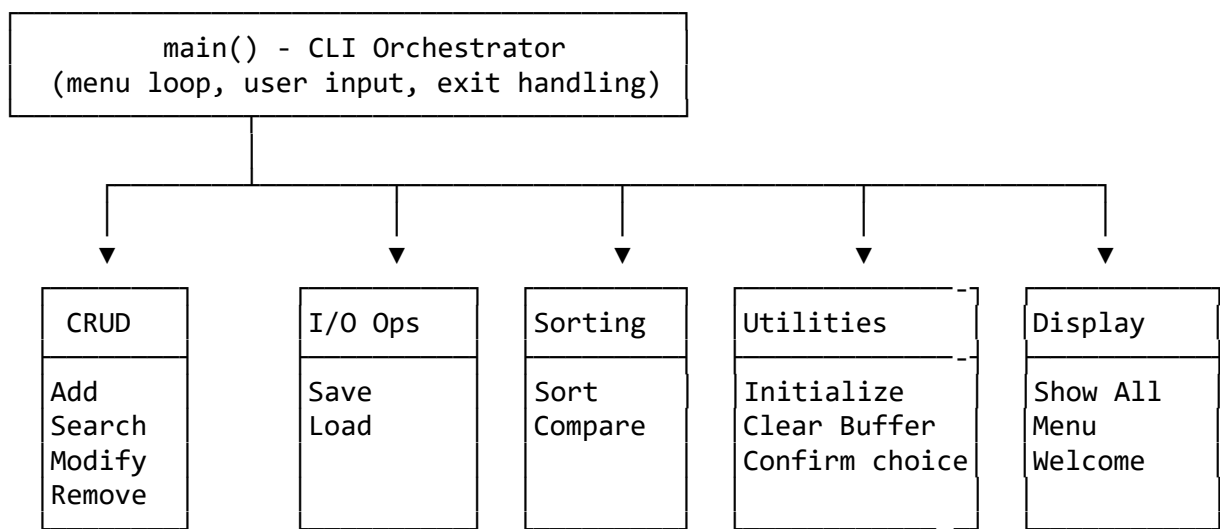
Calculations and Sorting

- Calculate and display the average marks for all students
- Provide an option to sort student records based on marks in ascending order
- Provide an option to sort student records based on marks in ascending or descending order.

3. Planning and Considerations

System Architecture

We designed this system using a **modular, function-based approach**. Each operation is a separate function, making the code maintainable and testable. The program follows a **client-server pattern** where the main function acts as an orchestrator that reads user input and delegates work to specialized functions.



Data Model

The system uses two main C structs:

Student Record:

```
typedef struct {  
    char name[50];           // Student's full name (max 50 chars)  
    int rollNumber;          // Unique identifier  
    float marks;              // Score out of 100  
} Student;
```

System State:

```
typedef struct {
    Student *students;    // Dynamic array of students
    int count;            // Number of students currently stored
    int capacity;          // Maximum students before resize
} StudentRecordSystem;
```

Key Design Decisions: - **Dynamic Array:** Starts with capacity 100; doubles when full (prevents fixed-size limitations) - **Pass/Fail Status:** Computed on-the-fly (not stored) as marks >= 40 ? "PASS" : "FAIL" - **Comma-separated text format (CSV-style):** For file storage (easy to view in Excel or a text editor, portable across systems) - **Input Validation:** Each function validates data before storage

Planned Core Features and Responsible Functions

During planning, we listed the main features we wanted and **which function** would implement each feature:

Planned feature	Core function(s)
Add student with validation	addStudent()
Display formatted table of students	displayAllStudents()
Search by roll number	searchStudent()
Modify name or marks	modifyStudent()
Remove student	removeStudent()
Calculate average marks	calculateAverageMarks()
Sort ascending/descending by marks	sortStudents()
Save records to comma-separated text file (CSV-style)	saveToFile()
Load records from comma-separated text file (CSV-style)	loadFromFile()
Dynamic array resizing when full	realloc() inside core functions
Pass/fail status display	displayAllStudents()
User confirmation before actions	confirmAction() / main()
CLI menu loop and navigation	main()

Planned Edge Case Considerations

We identified important **edge cases** that our functions should handle:

- 1. **Empty record set** – All display/analyze functions check if (count == 0) and print a friendly message.
- 2. **Duplicate roll numbers** – addStudent() prevents duplicate roll numbers.
- 3. **Array capacity exceeded** – realloc() automatically doubles capacity when needed.
- 4. **Invalid input** – Marks validated 0–100, roll numbers > 0, names non-empty.
- 5. **File I/O errors** – fopen() is checked; an error message is printed if the file cannot be opened.
- 6. **Malformed CSV-style lines** – strtok() results are validated; missing fields are skipped.
- 7. **Memory allocation failure** – malloc() / realloc() are checked; the program exits or aborts the operation with an error message.
- 8. **Partial modification** – The user can press Enter to skip a field and keep the old value.
- 9. **Character buffer overflow** – fgets() is used instead of gets(), so each read is limited to the size of the destination buffer and always NUL-terminated.
- 10. **Input buffer pollution** – clearInputBuffer() is called after scanf() to remove trailing newline characters.

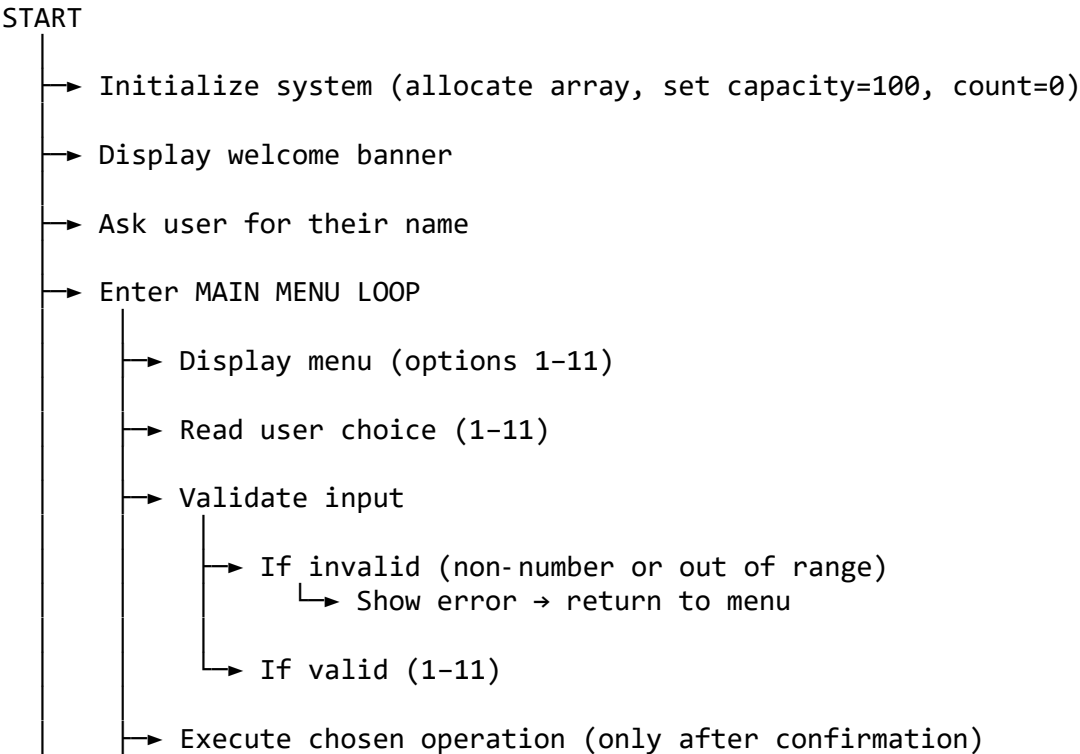
Code Review Strategy

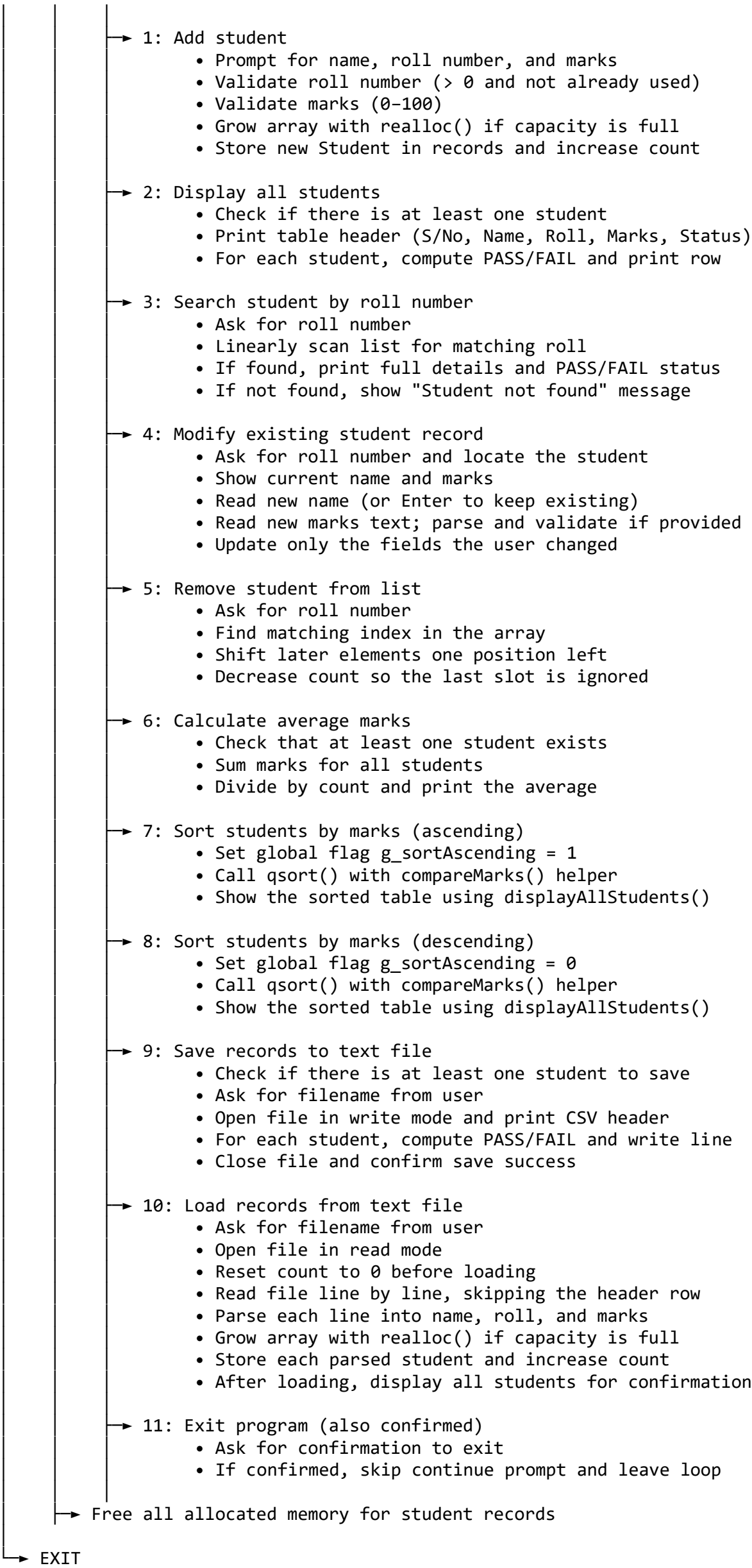
The team followed a **pair-based review model:** - Two developers implement the same function independently - Two shared reviewers compare both implementations and select the best - All team members studied the group leader’s main function to ensure architectural alignment - Quality criteria: correctness, error handling, code clarity, efficiency

4. Program Flow and User Story

Detailed Program Flow and Menu Loop

When the program starts, it follows this overall sequence. Note that **after a user selects a menu option, the program now asks for confirmation (y/Y to continue, n/N to cancel) before running the chosen action.** This protects against accidental operations like modifying or deleting records.



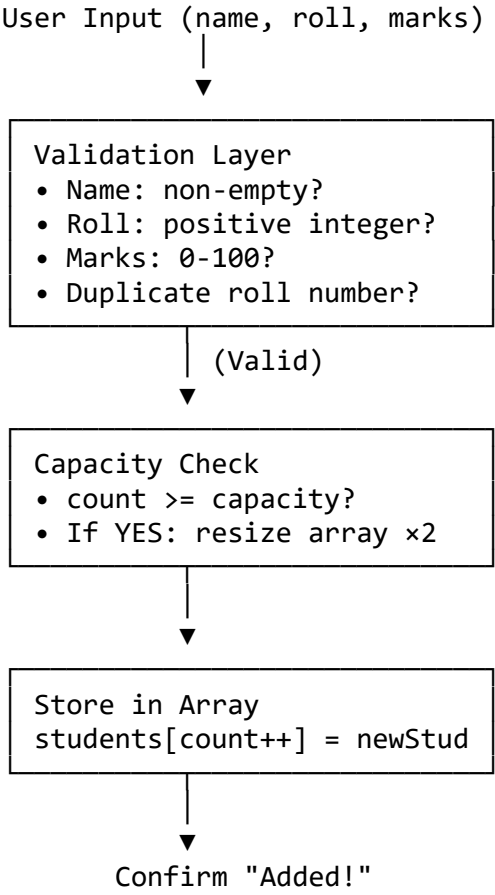


User Story: Adding and Sorting Students

Scenario: A teacher wants to manage a class of 50 students.

1. **Start Program** → Greets teacher, shows menu
2. **Add Students** → Teacher selects option 1, enters 50 students one by one
 - System validates each name (non-empty), roll number (positive, no duplicates), and marks (0-100)
 - Array auto-resizes from 100→100→100... (no resize needed in this case)
3. **Display All** → Teacher selects option 2, sees formatted table with pass/fail status
4. **Sort Descending** → Teacher selects option 8, students sorted by marks (highest first)
5. **Save to File** → Teacher selects option 9, saves to class_records.csv
6. **Exit** → Teacher selects option 11, program frees memory and closes

Data Flow Diagram: Adding a Student



5. Program Code

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_NAME_LENGTH 50
#define PASSING_MARKS 40

// student data structure
typedef struct {
    char name[MAX_NAME_LENGTH];
    int rollNumber;
    float marks;
} Student;

//Holds the entire collection of students and metadata about the array
typedef struct {
    Student *students;
    int count;
    int capacity;
} StudentRecordSystem;

// Function declarations
void initializeSystem(StudentRecordSystem *records);
void displayWelcomeMessage();
void addStudent(StudentRecordSystem *records);
void displayAllStudents(const StudentRecordSystem *records);
void searchStudent(const StudentRecordSystem *records);
void modifyStudent(StudentRecordSystem *records);
void removeStudent(StudentRecordSystem *records);
void calculateAverageMarks(const StudentRecordSystem *records);
void sortStudents(StudentRecordSystem *records, int ascending);
void saveToFile(const StudentRecordSystem *records);
void loadFromFile(StudentRecordSystem *records);
void displayMenu();
void clearInputBuffer();
int confirmAction(const char *actionDescription);

//main point of entry and runs the CLI Loop
int main() {

    StudentRecordSystem records;
    initializeSystem(&records);

    char userName[50];
    displayWelcomeMessage();
    printf("\nPlease enter your name: ");

    fgets(userName, sizeof(userName), stdin);
    userName[strcspn(userName, "\n")] = '\0'; // trim username and remove newline

    printf("\nHello, %s! Welcome to the Student Record System.\n\n", userName);

    //loop to display the menu and get user choice
    int choice;

    do {
        displayMenu();
        printf("Enter your choice (1-11): ");

        if (scanf("%d", &choice) != 1) {
            printf("Invalid input! Please enter a number.\n");
            clearInputBuffer();
        }
    } while (choice < 1 || choice > 11);
}
```



```

        continue;
    }
    clearInputBuffer();

    switch (choice) {
        case 1:
            if (!confirmAction("add a new student")) break;
            addStudent(&records);
            break;
        case 2:
            if (!confirmAction("display all students")) break;
            displayAllStudents(&records);
            break;
        case 3:
            if (!confirmAction("search for a student")) break;
            searchStudent(&records);
            break;
        case 4:
            if (!confirmAction("modify student records")) break;
            modifyStudent(&records);
            break;
        case 5:
            if (!confirmAction("remove a student")) break;
            removeStudent(&records);
            break;
        case 6:
            if (!confirmAction("calculate average marks")) break;
            calculateAverageMarks(&records);
            break;
        case 7:
            if (!confirmAction("sort students by marks in ascending order")) break;
            sortStudents(&records, 1); // Ascending
            break;
        case 8:
            if (!confirmAction("sort students by marks in descending order")) break;
            sortStudents(&records, 0); // Descending
            break;
        case 9:
            if (!confirmAction("save records to a file")) break;
            saveToFile(&records);
            break;
        case 10:
            if (!confirmAction("load records from a file")) break;
            loadFromFile(&records);
            break;
        case 11:
            if (confirmAction("exit the program")) {
                printf("Thank you for using the Student Record System, %s! Goodbye!\n", userName);
            } else {
                /* User cancelled exit; stay in the menu loop. */
                choice = 0;
            }
            break;
        default:
            printf("Invalid choice! Please try again.\n");
    }
    printf("\n");

    // After performing the chosen action, ask the user if they want to continue or exit. if the user presses n or N, exit the
    // program.
    if (choice != 11) {
        char contBuf[8];
        printf("Do you want to continue? (y/n): ");
        if (fgets(contBuf, sizeof(contBuf), stdin) != NULL) {
            if (contBuf[0] == 'n' || contBuf[0] == 'N') {
                choice = 11; // set to exit
                printf("Exiting as requested.\n");
            }
        }
    }
} while (choice != 11);

// Free dynamically allocated memory we created in initializeSystem.
free(records.students);

return 0;
}

void initializeSystem(StudentRecordSystem *records) {

    // Choose a small default capacity; realloc will grow if needed.
    records->capacity = 100;
    records->students = (Student*)malloc(records->capacity * sizeof(Student));
    records->count = 0;

    // Simple check: exit if heap allocation fails
    if (records->students == NULL) {
        printf("Memory allocation failed!\n");
        exit(1);
    }
}

void displayWelcomeMessage() {
    printf("=====\n");
    printf("      STUDENT RECORD SYSTEM\n");
    printf("=====\n");
}

```

```

void displayMenu() {
    printf("=====\n");
    printf("          MAIN MENU\n");
    printf("=====\n");
    printf("1. Add Student\n");
    printf("2. Display All Students\n");
    printf("3. Search Student by Roll Number\n");
    printf("4. Modify Student Record\n");
    printf("5. Remove Student\n");
    printf("6. Calculate Average Marks\n");
    printf("7. Sort Students by Marks (Ascending)\n");
    printf("8. Sort Students by Marks (Descending)\n");
    printf("9. Save Records to File\n");
    printf("10. Load Records from File\n");
    printf("11. Exit\n");
    printf("-----\n");
}

//function to add new student, their name roll number and marks, it also checks if the student already exists
void addStudent(StudentRecordSystem *records) {
    if (records->count >= records->capacity) {
        // Resize the array
        records->capacity *= 2;
        records->students = (Student*)realloc(records->students, records->capacity * sizeof(Student));
        if (records->students == NULL) {
            printf("Memory allocation failed!\n");
            return;
        }
    }

    Student newStudent;

    printf("\n--- Add New Student ---\n");

    printf("Enter student name: ");
    fgets(newStudent.name, sizeof(newStudent.name), stdin);
    newStudent.name[strcspn(newStudent.name, "\n")] = '\0';

    printf("Enter roll number: ");
    while (scanf("%d", &newStudent.rollNumber) != 1 || newStudent.rollNumber <= 0) {
        printf("Invalid roll number! Please enter a positive integer: ");
        clearInputBuffer();
    }
    clearInputBuffer();

    // Check if roll number already exists
    for (int i = 0; i < records->count; i++) {
        if (records->students[i].rollNumber == newStudent.rollNumber) {
            printf("Error: Roll number %d already exists!\n", newStudent.rollNumber);
            return;
        }
    }

    printf("Enter marks: ");
    while (scanf("%f", &newStudent.marks) != 1 || newStudent.marks < 0 || newStudent.marks > 100) {
        printf("Invalid marks! Please enter a value between 0 and 100: ");
        clearInputBuffer();
    }
    clearInputBuffer();

    records->students[records->count] = newStudent;
    records->count++;

    printf("Student added successfully!\n");
}

//function to display all students like a table, with appropriate spacing by using things like %-5s, %-12d etc.
void displayAllStudents(const StudentRecordSystem *records) {
    if (records->count == 0) {
        printf("No students in the record system.\n");
        return;
    }

    printf("\n--- All Students ---\n");
    printf("%-5s %-20s %-12s %-8s %s\n", "S.No", "Name", "Roll Number", "Marks", "Status");
    printf("-----\n");

    for (int i = 0; i < records->count; i++) {
        const Student *student = &records->students[i];
        char status[10];
        strcpy(status, (student->marks >= PASSING_MARKS) ? "PASS" : "FAIL");

        printf("%-5d %-20s %-12d %-8.2f %s\n",
            i + 1, student->name, student->rollNumber, student->marks, status);
    }
}

//function to search student by roll number and it checks if record is empty before searching
void searchStudent(const StudentRecordSystem *records) {
    if (records->count == 0) {
        printf("No students in the record system.\n");
        return;
    }

    int rollNumber;
    printf("\n--- Search Student ---\n");

```

```

printf("Enter roll number to search: ");

if (scanf("%d", &rollNumber) != 1) {
    printf("Invalid input!\n");
    clearInputBuffer();
    return;
}
clearInputBuffer();

for (int i = 0; i < records->count; i++) {
    if (records->students[i].rollNumber == rollNumber) {
        const Student *student = &records->students[i];
        char status[10];
        strcpy(status, (student->marks >= PASSING_MARKS) ? "PASS" : "FAIL");

        printf("\nStudent Found:\n");
        printf("Name: %s\n", student->name);
        printf("Roll Number: %d\n", student->rollNumber);
        printf("Marks: %.2f\n", student->marks);
        printf("Status: %s\n", status);
        return;
    }
}

printf("Student with roll number %d not found.\n", rollNumber);
}

//function to modify student by using their roll number
void modifyStudent(StudentRecordSystem *records) {
    if (records->count == 0) {
        printf("No students in the record system.\n");
        return;
    }

    int rollNumber;
    printf("\n--- Modify Student Record ---\n");
    printf("Enter roll number of student to modify: ");

    if (scanf("%d", &rollNumber) != 1) {
        printf("Invalid input!\n");
        clearInputBuffer();
        return;
    }
    clearInputBuffer();

    for (int i = 0; i < records->count; i++) {
        if (records->students[i].rollNumber == rollNumber) {
            printf("Student found. Current details:\n");
            printf("Name: %s\n", records->students[i].name);
            printf("Roll Number: %d\n", records->students[i].rollNumber);
            printf("Marks: %.2f\n", records->students[i].marks);

            printf("\nEnter new name (or press Enter to keep current): ");
            char newName[MAX_NAME_LENGTH];
            fgets(newName, sizeof(newName), stdin);
            newName[strcspn(newName, "\n")] = '\0';

            if (strlen(newName) > 0) {
                strcpy(records->students[i].name, newName);
            }

            printf("Enter new marks (or press Enter to keep current): ");
            char marksInput[32];
            if (fgets(marksInput, sizeof(marksInput), stdin) != NULL) {
                marksInput[strcspn(marksInput, "\n")] = '\0';
                if (strlen(marksInput) == 0) {
                    printf("No marks entered; keeping current marks.\n");
                } else {
                    char *endptr;
                    float newMarks = strtod(marksInput, &endptr);
                    if (endptr != marksInput && *endptr == '\0') {
                        if (newMarks >= 0 && newMarks <= 100) {
                            records->students[i].marks = newMarks;
                        } else {
                            printf("Invalid marks! Keeping current marks.\n");
                        }
                    } else {
                        printf("Invalid marks input! Keeping current marks.\n");
                    }
                }
            }

            printf("Student record updated successfully!\n");
            return;
        }
    }

    printf("Student with roll number %d not found.\n", rollNumber);
}

//function to remove student by roll number
void removeStudent(StudentRecordSystem *records) {
    if (records->count == 0) {
        printf("No students in the record system.\n");
        return;
    }
}

```



```

    int rollNumber;
    printf("\n--- Remove Student ---\n");
    printf("Enter roll number of student to remove: ");

    if (scanf("%d", &rollNumber) != 1) {
        printf("Invalid input!\n");
        clearInputBuffer();
        return;
    }
    clearInputBuffer();

    for (int i = 0; i < records->count; i++) {
        if (records->students[i].rollNumber == rollNumber) {
            // Shift all elements after the found student one position left
            for (int j = i; j < records->count - 1; j++) {
                records->students[j] = records->students[j + 1];
            }
            records->count--;
            printf("Student with roll number %d removed successfully!\n", rollNumber);
            return;
        }
    }

    printf("Student with roll number %d not found.\n", rollNumber);
}

//function to calculate average score of all students
void calculateAverageMarks(const StudentRecordSystem *records) {
    if (records->count == 0) {
        printf("No students in the record system.\n");
        return;
    }

    float total = 0;
    for (int i = 0; i < records->count; i++) {
        total += records->students[i].marks;
    }

    float average = total / records->count;
    printf("\n--- Average Marks ---\n");
    printf("Total students: %d\n", records->count);
    printf("Average marks: %.2f\n", average);
}

//global variable to indicate ascending or descending sort
static int g_sortAscending = 1;

//helper function for qsort to compare marks, we need this function to
//tell qsort what field to sort on and in which direction
static int compareMarks(const void *a, const void *b) {
    // The comparator returns negative/zero/positive to indicate ordering to qsort.
    const Student *sa = (const Student *)a;
    const Student *sb = (const Student *)b;

    if (sa->marks < sb->marks) return g_sortAscending ? -1 : 1;
    if (sa->marks > sb->marks) return g_sortAscending ? 1 : -1;
    return 0; // equal marks
}

//function to sort students by marks in ascending or descending order using qsort
void sortStudents(StudentRecordSystem *records, int ascending) {
    if (records->count == 0) {
        printf("No students in the record system.\n");
        return;
    }

    // Set the global flag then call qsort with the single comparator.
    g_sortAscending = ascending ? 1 : 0;
    qsort(records->students, records->count, sizeof(Student), compareMarks);

    printf("Students sorted by marks in %s order.\n", ascending ? "ascending" : "descending");
    displayAllStudents(records);
}

//function to save to file with proper file handling and CSV format
void saveToFile(const StudentRecordSystem *records) {
    if (records->count == 0) {
        printf("No students to save.\n");
        return;
    }

    char filename[100];
    printf("\n--- Save to File ---\n");
    printf("Enter filename to save records: ");
    fgets(filename, sizeof(filename), stdin);
    filename[strcspn(filename, "\n")] = '\0';

    FILE *file = fopen(filename, "w");
    if (file == NULL) {
        printf("Error opening file for writing!\n");
        return;
    }

    // Write header
    fprintf(file, "Name,RollNumber,Marks,Status\n");

    // Write student data

```

```

    for (int i = 0; i < records->count; i++) {
        const Student *student = &records->students[i];
        char status[10];
        strcpy(status, (student->marks >= PASSING_MARKS) ? "PASS" : "FAIL");

        fprintf(file, "%s,%d,%.2f,%s\n",
            student->name, student->rollNumber, student->marks, status);
    }

    fclose(file);
    printf("Student records saved to '%s' successfully!\n", filename);
}

void loadFromFile(StudentRecordSystem *records) {
    char filename[100];
    printf("\n--- Load from File ---\n");
    printf("Enter filename to load records: ");
    fgets(filename, sizeof(filename), stdin);
    filename[strcspn(filename, "\n")] = '\0';

    FILE *file = fopen(filename, "r");
    if (file == NULL) {
        printf("Error opening file for reading!\n");
        return;
    }

    // Clear existing records
    records->count = 0;

    char line[256];
    int lineNumber = 0;

    // Read file line-by-line. CSV format assumed: Name,RollNumber,Marks,Status
    while (fgets(line, sizeof(line), file)) {
        lineNumber++;

        // Skip header line
        if (lineNumber == 1) continue;

        // Remove newline character
        line[strcspn(line, "\n")] = '\0';

        // Parse CSV line
        char *token = strtok(line, ",");
        if (token == NULL) continue;

        if (records->count >= records->capacity) {
            records->capacity *= 2;
            records->students = (Student*)realloc(records->students, records->capacity * sizeof(Student));
            if (records->students == NULL) {
                printf("Memory allocation failed!\n");
                fclose(file);
                return;
            }
        }

        strcpy(records->students[records->count].name, token);

        token = strtok(NULL, ",");
        if (token == NULL) continue;
        records->students[records->count].rollNumber = atoi(token);

        token = strtok(NULL, ",");
        if (token == NULL) continue;
        records->students[records->count].marks = atof(token);

        records->count++;
    }

    fclose(file);
    printf("Student records loaded from '%s' successfully! (%d students loaded)\n",
        filename, records->count);

    // If any valid records were loaded, display them immediately so the user can confirm contents right away.
    if (records->count > 0) {
        displayAllStudents(records);
    }
}

int confirmAction(const char *actionDescription) {
    char response[8];
    printf("Are you sure you want to %s? Press y/Y to continue, n/N to cancel: ", actionDescription);
    if (fgets(response, sizeof(response), stdin) == NULL) {
        printf("Input error. Cancelling action.\n");
        return 0;
    }
    if (response[0] == 'y' || response[0] == 'Y') {
        return 1;
    }
    printf("Action cancelled.\n");
    return 0;
}

void clearInputBuffer() {
    int c;
    // Consume remaining chars up to newline so subsequent fgets() works.

```

```
    while ((c = getchar()) != '\n' && c != EOF);  
}
```

6. Quick Code Explanation

This is a high-level Explanation the code

- **Main structure:** The program defines a `Student` struct (name, roll number, marks) and a `StudentRecordSystem` struct that holds a dynamic array of students plus count and capacity.
- **Initialization:** `initializeSystem()` allocates an initial array of 100 students and makes sure memory allocation succeeds before the program continues.
- **User interface:** `main()` shows a text menu in a loop. Based on the user's choice (1–11) it first asks for confirmation using a small helper (`confirmAction()`), and only if the user answers y/Y does it call the functions to add, display, search, modify, remove, sort, save, or load students, or to exit the program.
- **Adding and viewing students:** `addStudent()` collects validated input (name, unique positive roll number, marks between 0 and 100). `displayAllStudents()` prints a neat table and shows PASS or FAIL depending on whether marks are at least 40.
- **Searching and editing:** `searchStudent()` looks up a student by roll number and prints their details. `modifyStudent()` lets the user change name and/or marks while keeping other fields the same.
- **Deleting and statistics:** `removeStudent()` deletes a student by shifting later records left and reducing count. `calculateAverageMarks()` loops through all students and prints the class average.
- **Sorting:** A global flag `g_sortAscending` and the helper `compareMarks()` are used with `qsort()` so `sortStudents()` can sort by marks in either ascending or descending order, then immediately display the sorted list.
- **File operations:** `saveToFile()` writes all records as CSV (including computed pass/fail status). `loadFromFile()` reads CSV lines back, dynamically growing the array as needed and then displaying the loaded data.
- **Input safety:** `clearInputBuffer()` cleans up leftover characters after `scanf()`, and `fgets()` is used for strings to avoid buffer overflows and mixed input problems.

8. Conclusion

The **Student Record System** is a robust, modular C application that demonstrates core programming concepts: dynamic memory management, input validation, file I/O, sorting algorithms, and user interface design.

What We Learned

1. **Modular Design:** Separating concerns (CRUD, I/O, sorting, UI) makes code maintainable and testable.
2. **Error Handling:** Validating input and checking return values (`malloc`, `fopen`) prevents crashes.
3. **User Experience:** Clear prompts, formatted output, and graceful handling of edge cases build trust.
4. **Memory Safety:** Using `fgets()` over `gets()`, checking buffer sizes, and freeing memory prevents vulnerabilities.
5. **Standard Library Functions:** `qsort()`, `strtok()`, `realloc()` are powerful tools when used correctly:
 - `qsort()` – generic sort function from `<stdlib.h>` that can sort any array when you provide the element size and a comparison function (we use it to sort students by marks).
 - `strtok()` – tokenizes a string based on a delimiter (, in our CSV lines), letting us split each line into name, roll number, and marks.
 - `realloc()` – resizes an existing heap allocation while preserving current data, which we use to grow the student array when capacity is exceeded.

Future Enhancements

- **Sorting by name or roll number** (not just marks)
- **Grade statistics** (min, max, median, standard deviation)
- **Batch operations** (import 1000+ records from CSV)